

Rogue-like Algebra

Lab 1: Rogue-like Algebra

Gabriel A. Lugo-Maldonado

CS 411W

Thomas Kennedy

04 September 2025

Version 1

Table of Contents

1 Introduction	2
2 Rogue-like Algebra.....	2
3 Identification of Case Study	4
4 Glossary	6
5 References	8

Table of Figures

No table of figures entries found.

1 Introduction

Algebra is a very important skill that may be utilized throughout everyday life. It is used across many disciplines, such as carpentry, art, construction, and STEM fields. Despite this, according to the U.S. Department of Education's PIAAC long-term study, the average U.S. adult numeracy skills have been deteriorating over the last decade. Many people struggle to understand and utilize this fundamental skill.

This struggle may be due to several reasons, but a dominant reason may be the mundaneness of traditional teaching methods. Students, and adults interested in learning, are lacking actual engagement with the material. This results in students failing to retain knowledge and eventually frustration and failure.

To solve this societal problem, a game shall be proposed. Games are powerful tools for indirectly influencing a person's way of thinking. They require the player to be involved in the action unfolding in front of them, and many require them to constantly think of better ways to accomplish a task. Some popular games such as *Minecraft* demonstrate this well. The "redstone" system teaches players simple electrical engineering skills without being overly explanatory.

This indirect learning method is typically much more engaging and fun for the average person; therefore, it should be at the center of the solution to the societal struggle. Rogue-like Algebra will be such a game.

2 Rogue-like Algebra

Rogue-like Algebra will be a single-player, turn-based, rogue-like, action-adventure game. Players will be challenged to trek through procedural dungeons, solving increasingly difficult problems to make progress. Players will be rewarded with things such as upgrades and enhancements for correct answers, but incorrect answers will lead to lost health and eventually permadeath.

2.1 Key Product Features and Capabilities

As mentioned in Section 2, Rogue-like Algebra will feature turn-based combat to allow players time to think and decide on their next answer. Progressive difficulty, time challenges, and periodic “boss battles” will mix up gameplay and provide forward pressure on the players. Minigames will allow players to not feel that the game is getting too repetitive while also allowing the development team to provide various opportunities to learn different topics in algebra. Immersive storylines will additionally pull in additional players and keep them engaged. The drive to know what comes next in the story may push students to try as hard as they can to keep going until the end.

The game will be quite innovative in that it will prioritize gameplay rather than learning. This may seem counter-intuitive, but the reasoning is that a purely educational game may not be as engaging as one that features truly entertaining gameplay. Furthermore, gameplay that features learning as something done as a secondary objective is a common great way to transform an otherwise boring lesson plan into a fun adventure. This solution will push students to keep learning and practicing on their own even outside of class time.

2.2 Major Components

At the heart of the development of the game, the team will use the Godot game engine. This choice was very straightforward as the engine is free, open-source, and quite extensible. The engine also has a massive community following with plenty of documentation, tutorials, and readily available pre-made components. To go along with Godot, Rogue-like Algebra will be created primarily using the Rust programming language. Rust is a fast, modern, and memory-safe language which is becoming increasingly popular in the game development circles. GDScript, Godot’s default built-in scripting language, may need to be used for certain edge cases which are not supported by the Rust language bindings. The target for distribution will be a web-based application, allowing it to run anywhere a web-browser can.

Visual Studio Code will be used as the editor of choice as, like Godot, it is free and open source. This means the team will not have any issues using it for commercial purposes should the game be released as a monetized product. Version control shall be accomplished via GitHub. GitHub is a staple in the software development industry, so this should not be a surprise. Since GitHub will be used for version control, GitHub Actions and Projects will additionally be used to steer development forwards with features such as continuous integration and kanban boards.

Procedural generators for problems to ask the player and for dungeon layouts will be required. These are core to the gameplay as without them, there is not much replay value and therefore little to drive students to keep practicing. Character controllers will need to be created or adapted for use in the game. Assuming a top-down perspective is chosen, this perspective will be quite easy to adapt from freely available templates and tutorials posted online. The biggest challenge will be creating fun and engaging enemy AI. In this context, “AI” refers to non-player character decision-making. AI is quite a behemoth of a task to implement in any game, so it will be expected to be a point of focus for balanced gameplay.

3 Identification of Case Study

The target audience of Rogue-like Algebra is majorly twofold: a primary audience of middle and high schoolers, aged 12 – 18, who are learning algebra for the first time through educational institutions; but also, a secondary audience of players interested in STEM topics who may not be in school anymore. There will inevitably be an additional tertiary major audience of those interested in indie games and rogue-likes; however, these will not be part of the intended target audience.

For the primary audience, the purpose of the game is to assist students in topic digestion and retention. By allowing students to approach the topics from an alternative perspective in a world they might be more comfortable with, i.e. gaming, the hope is that they may be more likely to both comprehend and retain the complex mathematics more quickly.

And with the secondary audience, the game will provide an excellent introduction or refresher, depending on the incoming skill of the player. Through layers of indirection via abstraction layers over the math, players will learn without feeling boredom and dread. The fun, replayable nature of the game and light-hearted story should allow for players to be sucked in despite not being required to play it by any external forces. This is key to keeping the attention of the secondary audience.

Future audiences may include the tertiary audience mentioned at the start of this section, but also companies looking to retrain staff and perhaps teachers looking to challenge students as a part of extracurricular activities. There is a lot of potential for games that excel at educating the masses.

4 Glossary

Adaptive Learning – An educational method that adjusts content difficulty based on the learner’s performance to personalize the learning experience.

Algebraic Concepts – Foundational mathematical principles involving variables, equations, expressions, and functions typically taught in middle and high school.

Continuous Integration (CI) – A development practice where developers frequently merge code changes into a shared repository, triggering automated builds and tests to detect issues early.

Continuous Deployment (CD) – A software release process that automatically delivers validated code changes to production, minimizing manual intervention and ensuring faster updates.

GDScript – A Python-like scripting language used in the Godot game engine, designed for creating game logic.

Git – A distributed version control system that tracks changes in source code during software development, enabling collaboration and history tracking.

GitHub – A cloud-based platform for hosting Git repositories. It offers tools for collaboration, issue tracking, code review, and CI/CD integration.

Godot – A free and open-source game engine used for developing 2D and 3D games, known for its flexibility and ease of use.

IDE (Integrated Development Environment) – A software suite that provides tools like a code editor, debugger, and compiler to streamline software development (e.g., VS Code).

Procedural Generation – A method of creating data algorithmically rather than manually, often used for generating levels, puzzles, or content dynamically in games.

Rogue-like – A subgenre of games typically featuring procedural generation, turn-based movement, and permanent death, emphasizing strategy and replayability.

Rust – A systems programming language known for performance, safety, and concurrency.

Turn-Based Combat – A gameplay mechanic where players and opponents take alternating turns to perform actions such as attacking or defending.

VS Code (Visual Studio Code) – A popular, lightweight code editor developed by Microsoft, used for writing and debugging code.

5 References

“About Us - DragonBox / WeWantToKnow.” Dragonbox.com, dragonbox.com/about.

Prodigy. “Prodigy Math | Prodigy Education.” Www.prodigygame.com, www.prodigygame.com/main-en/prodigy-math/.

“IXL - Membership.” Www.ixl.com, www.ixl.com/membership.

Steam. “Steam Database.” SteamDB, steamdb.info/.

“Top Roguelike Games by Sales and Revenue on Steam as of 2022 – Steam Marketing Tool.” Games-Stats.com, games-stats.com/steam/?tag=roguelike.

Decker-Woodrow, Lauren E, et al. The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19. Vol. 9, 1 Jan. 2023, p. 233285842311659-233285842311659, <https://doi.org/10.1177/23328584231165919>.

Aine, Sandip, and Maxim Likhachev. “Anytime Truncated D* : Anytime Replanning with Truncation.” Proceedings of the International Symposium on Combinatorial Search, vol. 4, no. 1, 20 Aug. 2021, pp. 2–10, <https://doi.org/10.1609/socs.v4i1.18295>

Guy, Stephen, and Ioannis Karamouzas. Guide to Anticipatory Collision Avoidance.

“NavigationMesh.” Godot Engine Documentation, 2025, docs.godotengine.org/en/stable/classes/class_navigationmesh.html

“Creating Your First 2D Game with A* Algorithm.” HackerEarth Blog, 6 Oct. 2016, hackerearth.com/blog/developers/creating-first-2d-game-algorithm/

U.S. Department of Education, National Center for Education Statistics. (2024). *Highlights of the 2023 U.S. PIAAC Results Web Report* (NCES 2024-202). Washington, DC. Retrieved 2 May 2025 from https://nces.ed.gov/surveys/piaac/2023/national_results.asp.

Minecraft, Mojang Studios, www.minecraft.net/en-us. Accessed 2 May 2025.